

Reporte de práctica: Crear, copiar, mover y eliminar archivos y carpetas

1. Introducción

La terminal de Ubuntu permite interactuar directamente con el sistema operativo mediante comandos para administrar archivos y directorios. A través de estas instrucciones es posible crear carpetas, generar archivos, copiar información entre diferentes ubicaciones y eliminar elementos del sistema de archivos.

En esta práctica se realizaron diferentes operaciones utilizando los comandos `mkdir`, `ls`, `cp`, `cp -r`, `rm` y `rm -r`. Además de observar el funcionamiento de cada comando, se analizaron las rutas utilizadas y el comportamiento del sistema operativo al realizar las operaciones solicitadas.

2. Objetivo

Analizar el funcionamiento de comandos básicos de Ubuntu para la administración de archivos y directorios, comprendiendo el comportamiento del sistema operativo ante operaciones de creación, consulta, copia y eliminación, así como la importancia del uso correcto de rutas y permisos.

3. Desarrollo de la práctica

3.1 Creación de directorios

La práctica comenzó creando dos directorios dentro de la carpeta `Documents`:

```
mkdir practica1  
mkdir practica2
```

El comando `mkdir` permite crear nuevos directorios en el sistema de archivos. Como resultado, se generaron las carpetas `practica1` y `practica2`, las cuales fueron utilizadas posteriormente como directorios de origen y destino para las diferentes operaciones.

Desde el punto de vista del sistema operativo, la creación de un directorio implica registrar una nueva entrada dentro del sistema de archivos y asignarle sus atributos correspondientes.

3.2 Creación y consulta de un archivo

Posteriormente se ingresó a la carpeta `practica1` y se creó el archivo `Readme.txt` utilizando:

```
sudo gedit Readme.txt
```

Después de guardar el archivo se utilizó:

```
ls -l
```

La opción `-l` permite visualizar información detallada de los elementos del directorio, incluyendo permisos, propietario, grupo, tamaño y fecha de modificación.

Durante la práctica se pudo observar que el archivo creado mediante `sudo gedit` quedó asociado al usuario `root`. Esto se debe a que el programa fue ejecutado con privilegios administrativos mediante `sudo`. Esta observación permite comprender que los permisos y la propiedad de los archivos dependen del usuario con el que se ejecuta una operación.

4. Análisis de la copia de archivos

Para copiar el archivo `Readme.txt` desde `practica1` hacia `practica2`, se utilizó:

```
cp Readme.txt ../practica2/
```

El comando `cp` recibe una ubicación de origen y una ubicación de destino. En este caso, el archivo de origen era `Readme.txt` y el destino correspondía a la carpeta `practica2`.

La expresión `..` representa el directorio padre del directorio actual. Como la terminal se encontraba dentro de:

```
Documents/practica1/
```

al utilizar `..` se regresó a:

```
Documents/
```

y posteriormente se accedió a:

```
Documents/practica2/
```

Por lo tanto, la ruta utilizada fue una ruta relativa. El resultado permitió comprobar que no es necesario escribir siempre una ruta completa para realizar operaciones sobre archivos; también es posible utilizar la ubicación actual como referencia.

4.1 Análisis del error de ruta

Durante la práctica se intentó inicialmente realizar la copia mediante:

```
cp Readme.txt /practica2
```

El sistema operativo mostró un mensaje de error relacionado con los permisos.

Este comportamiento ocurrió porque `/practica2` no representa la carpeta `practica2` creada dentro de `Documents`. El símbolo `/` al inicio de una ruta representa el directorio raíz del sistema de archivos. Por lo tanto, el comando intentaba acceder a una ubicación diferente de la que se había creado para la práctica.

Posteriormente se corrigió la ruta utilizando:

```
cp Readme.txt ../practica2/
```

La operación se realizó correctamente.

Esta situación permitió comprobar la importancia de distinguir entre una ruta absoluta y una ruta relativa. Una ruta incorrecta puede provocar que el sistema operativo intente acceder a una ubicación diferente o que la operación sea rechazada por las restricciones de permisos.

5. Análisis de la copia de directorios

Para copiar directorios completos se utilizó la opción recursiva `-r` del comando `cp`.

Primero se copió la carpeta `Vacia`:

```
cp -r ~/Documents/practica2/Vacia ~/Documents/practical
```

Posteriormente se copió la carpeta `info`:

```
cp -r ~/Documents/practica2/info ~/Documents/practical
```

La opción `-r` permite que `cp` procese el directorio y su contenido de forma recursiva. Esto es necesario porque un directorio puede contener otros archivos y subdirectorios.

En el caso de `Vacia`, se comprobó el comportamiento al copiar un directorio sin archivos. En el caso de `info`, se observó que la operación también puede conservar el contenido existente dentro del directorio.

Esto demuestra que la copia de un directorio no se limita a crear una carpeta con el mismo nombre, sino que puede involucrar el procesamiento de los elementos contenidos dentro de ella.

6. Análisis de la eliminación de archivos

Para eliminar el archivo `Readme.txt` se utilizó:

```
rm Readme.txt
```

El comando `rm` permite eliminar archivos desde la terminal. A diferencia de una eliminación realizada mediante un entorno gráfico que puede enviar un elemento a la papelera, `rm` realiza directamente la operación solicitada sobre el sistema de archivos.

Por esta razón, el uso de este comando requiere verificar previamente que el nombre y la ubicación del archivo sean correctos.

7. Análisis de la eliminación de directorios

Finalmente se utilizó:

```
rm -r info/
```

La opción `-r` permite eliminar un directorio de forma recursiva junto con los elementos que contiene.

Esta operación es diferente a utilizar `rm` sobre un archivo individual, ya que el sistema debe procesar primero los elementos contenidos dentro del directorio para poder eliminar la estructura correspondiente.

El uso de `rm -r` también demuestra la importancia de comprobar las rutas antes de ejecutar comandos de eliminación, debido a que una ruta incorrecta podría provocar la eliminación de elementos diferentes a los deseados.

8. Análisis del comportamiento del sistema operativo

Las actividades realizadas permitieron observar que el sistema operativo interpreta cada comando de acuerdo con la ruta, los argumentos y los permisos proporcionados.

Las operaciones de creación mediante `mkdir` modificaron la estructura de directorios, mientras que `cp` permitió generar una copia de un archivo en otra ubicación. La utilización de `cp -r` permitió trabajar con estructuras de directorios completas.

Por otro lado, `rm` y `rm -r` modificaron nuevamente la estructura del sistema de archivos al eliminar los elementos indicados.

Uno de los aspectos más importantes observados fue la diferencia entre las rutas absolutas y relativas. El error producido al utilizar `/practica2` mostró que una ruta aparentemente sencilla puede hacer referencia a una ubicación completamente diferente. La corrección mediante `../practica2/` permitió comprobar cómo la posición actual de la terminal puede utilizarse para construir rutas relativas.

También se observó la relación entre los comandos ejecutados y los permisos del sistema. El uso de `sudo` para abrir `gedit` provocó que el archivo creado quedara asociado al usuario `root`, demostrando que las operaciones realizadas con privilegios administrativos pueden afectar la propiedad de los archivos.

9. Justificación teórica de las observaciones

El comportamiento observado durante la práctica corresponde al modelo de administración de archivos de los sistemas Linux. Los archivos y directorios se organizan jerárquicamente a partir de un directorio raíz representado por `/`.

Dentro de esta estructura, `.` representa el directorio actual y `..` representa el directorio padre. Esto explica el funcionamiento de la ruta `../practica2/` utilizada durante la copia del archivo.

Asimismo, Linux utiliza permisos y propietarios para controlar el acceso a los elementos del sistema de archivos. La información mostrada mediante `ls -l` permite observar parte de estos atributos. La ejecución de un programa mediante `sudo` utiliza privilegios administrativos, por lo que las operaciones realizadas de esta manera pueden generar archivos pertenecientes al usuario `root`.

Los comandos de copia y eliminación también interactúan directamente con el sistema de archivos. `cp` crea una copia del elemento indicado, mientras que `cp -r` permite procesar estructuras de directorios de manera recursiva. De forma similar, `rm` elimina archivos y `rm -r` permite eliminar directorios junto con su contenido.

10. Conclusión

La práctica permitió analizar el funcionamiento de diferentes comandos de Ubuntu utilizados para administrar archivos y directorios. Más allá de conocer la sintaxis de cada comando, las actividades permitieron observar cómo el sistema operativo interpreta las rutas, controla los permisos y modifica la estructura del sistema de archivos.

El error producido al utilizar `/practica2` permitió comprender la diferencia entre una ruta absoluta y una ruta relativa, mientras que el uso de `../practica2/` mostró cómo puede utilizarse el directorio padre para acceder a otra ubicación.

También se comprobó la utilidad de `cp -r` para trabajar con directorios y la diferencia entre `rm` y `rm -r` al eliminar archivos y carpetas. Finalmente, la utilización de `sudo` permitió observar la relación entre los privilegios administrativos y la propiedad de los archivos.

En conjunto, las actividades ayudaron a comprender de manera práctica algunos mecanismos básicos mediante los cuales Ubuntu organiza y administra la información almacenada en su sistema de archivos.